S2.01 : *Les Mystères de BUTopia*

Table des matières

1 SAÉ S2.01 Développement d'une application – résumé **5**
- 1.1 En bref . . . 5
- 1.2 Présentation . . . 5
- 1.3 Travail à faire . . . 6

2 Tutoriel de développement **7**
- 2.1 Lecture du code . . . 7
- 2.2 Analyse du code . . . 9
- 2.3 Premières modifications . . . 9
- 2.4 Batterie de feuilles . . . 10
- 2.5 Un premier dialogue . . . 11
- 2.6 Tests basiques . . . 11
- 2.7 Remise du tutoriel . . . 12

3 Développement guidé **13**
- 3.1 Édition basique . . . 13
- 3.2 Génération fixe . . . 14
- 3.3 Correction d'affichage . . . 14
- 3.4 Ajout : niveaux de compétence . . . 14
- 3.5 Épreuves et difficulté . . . 15
- 3.6 Résultats des épreuves, version simple et tests . . . 16
- 3.7 Interface d'épreuve . . . 16
- 3.8 Ergonomie : glisser-déposer . . . 16
- 3.9 Récapitulatif . . . 16

4 Développement décrit **17**
- 4.1 Cohérence des données . . . 17
- 4.2 Correction d'ergonomie . . . 17
- 4.3 Configurer quelques variables . . . 17
- 4.4 Enchaîner les épreuves . . . 17

4.5 Entraînement et expérience 18
4.6 Plans, sphères et compétences 18
4.7 Épreuves et compétences : compétences absentes 18
4.8 Épreuves et compétences : modificateurs de total 19
4.9 Épreuves et compétences : résultats automatiques 19
4.10 Épreuves et compétences : modificateurs de jet 19
4.11 Épreuves et compétences : tests de la version complète 20
4.12 Qualité de code, commentaires de documentation 20
4.13 Récapitulatif 20

5 Développement induit 21
5.1 Processus de création 21
5.2 Données supplémentaires et accessoires 21
5.3 Meilleurs visuels 21
5.4 Chargement des données depuis des fichiers 21
5.5 Points de talent 22
5.6 Santé mentale 22
5.7 Restructuration code, interface et qualité 22
5.8 L'application terminée 22

6 Développement objets : le dossier 23
6.1 Consignes 23
6.2 Les diagrammes initiaux 23
6.3 Le diagramme final 24
6.4 Le dictionnaire des responsabilités 24
6.5 Qualité : code, tests, documentation 24

7 Le compte-rendu en deux parties 25
7.1 Consignes 25
7.2 Compte-rendu commun 25
7.3 Compte-rendu individuel 25

8 Récapitulatif : livrables, dates, évaluations 27

A C# : rappels et compléments 29
A.1 Visibilités en C# et UML 29
A.2 Propriétés en C# et en UML 29
A.3 Valeur par défaut des paramètres 31
A.4 Opérateur ternaire 31
A.5 Abréviations 31
A.6 Pré-appels dans les constructeurs 32

A.7 Tuples 32
A.8 Les enum en C# 33
A.9 Méthodes d'extension 33
A.10 is, as 33

B Les Mystères de BUTopia **35**
B.1 BUTopia 35
B.2 Les Mystères 36
B.3 Détails du système 36

C Feuille à remplir **43**

Chapitre 1

SAÉ S2.01 Développement d'une application – résumé

1.1 En bref

La SAÉ S2.01 regroupe trois thématiques : développement d'une application avec interface humain-machine graphique, développement orienté objets, développement de qualité. En 2026, elle est mise en œuvre sur un projet en petite équipe : trois personnes, éventuellement deux ou quatre avec accord explicite de l'équipe pédagogique ou en solo sur un aménagement. La thématique est la gestion de feuilles de personnages pour un jeu de rôles fictif. Le rendu sera évalué sur plusieurs aspects, l'évaluation incluant également le report de notes de DS, et surtout un DS consacré à cette SAÉ. Le sujet sera uniquement distribué en français.

1.2 Présentation

L'objectif est de gérer des *fiches de personnages*, destinées à présenter certaines informations sur des personnages (fictifs) évoluant dans un monde (fictif) – il s'agit d'un paradigme courant pour les *jeux de rôles* sur table ou certains jeux vidéo, et pas si éloigné que ça des CV (d'un certain point de vue). Il s'agit donc de créer ces personnages, ou au moins leurs fiches ; ces fiches contiennent des valeurs appelées *indicateurs clés de performance* (indicateurs) et *évaluations critériées de compétences* (compétences) associant un intitulé (texte) et un niveau (numérique).

L'application doit permettre de : visualiser ces fiches, les éditer (modifier) et effectuer quelques opérations avec : un passage d'épreuves simulées, dont la réussite dépend des valeurs dans la fiche, et un entraînement simulé, qui permet de les améliorer.

Dans la réalisation de l'application, trois aspects sont à mener de front : les fonctionnalités, l'interface humain-machine, et la qualité de code (y compris tests et documentation).

Le code fourni est en anglais, comme les commentaires de documentation, fournis pour chaque élément public. Par contre, l'application et ses affichages sont en français.

Pour une présentation contextualisée (fictive), voir en Annexe B, p. 35. Une maquette de feuille de personnage est disponible en Annexe C, p. 43.

1.3 Travail à faire

Le développement de l'application est à faire en plusieurs stades.

1. Stade 0 : base – l'application qui vous est fournie.
2. Stade 1 : tutoriel – cette partie est guidée pas à pas, vos enseignants peuvent vous aider. Voir Chapitre 2, p. 7.

   À titre indicatif, le tutoriel devrait être terminé **dans les séances encadrées**.
3. Stade 2 : premières fonctionnalités – cette partie est assez guidée. Voir Chapitre 3, p. 13.

   À titre indicatif, cette partie devrait être terminée **la première semaine**.
4. Stade 3 : application complète utilisable – partie avec éléments que vous devrez interpréter. Voir Chapitre 4, p. 17.

   À titre indicatif, cette partie devrait **vous prendre la plus grande partie du temps de développement**.
5. Stade 4+ : finalisation et extensions possibles – plus libre. Voir Chapitre 5, p. 21.

   N'y consacrez du temps **que s'il vous en reste après avoir parfaitement fini les parties précédentes**.

Un autre travail vous sera demandé : le dossier de développement objets comprenant diagrammes et préparatifs de tests, voir Chapitre 6, p. 23. Enfin, nous demandons un compte-rendu rapide en deux parties : une commune à l'équipe, une personnelle. Voir Chapitre 7, p. 25.

Le travail est à faire :

— Tutoriel et réponses aux questions : très rapidement, rendu avant le vendredi 3 avril, 18:00,
— Tout le reste du travail : rendu avant le lundi 27 avril, 8:00,
— et il y aura aussi un DS de SAÉ le mercredi 22 avril à 16:00.

L'ensemble des livrables avec dates de rendu et les coefficients d'évaluation de cette SAÉ est disponible en Chapitre 8, p. 27.

Pour vous aider, vous trouverez quelques compléments et rappels sur le langage C# et UML en Annexe A, p. 29.

Chapitre 2

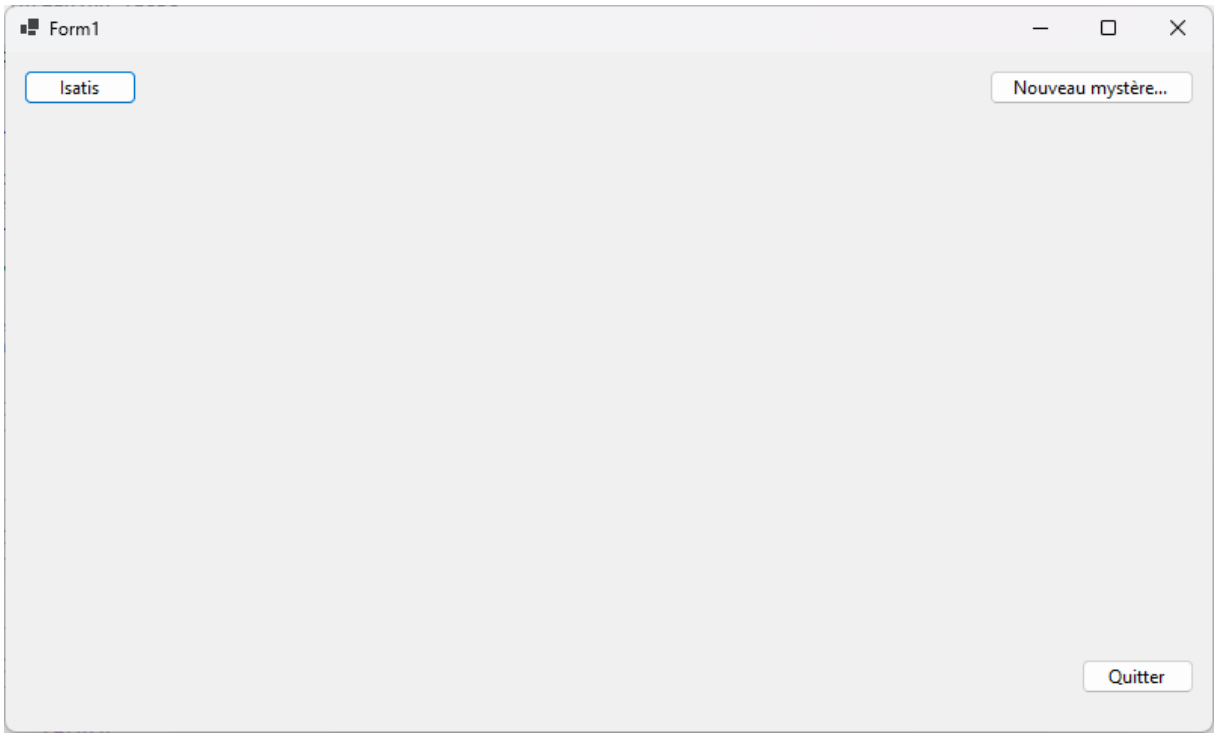
Tutoriel de développement

Pour le tutoriel, nous demandons à **chaque personne dans l'équipe** de le suivre intégralement pour que tout le monde intègre la structure. Vous êtes en droit de discuter **au sein de l'équipe** pour partager votre compréhension et vos réponses. Téléchargez le code fourni, parcourez-le ; il y a beaucoup d'éléments, suivez les instructions pas à pas. Chaque chose à faire est un **exercice** numéroté (un seul système de numérotation). Pour chaque **question** dans ce tutoriel, notez la réponse dans un fichier texte. Pour chaque **dossier**, le résultat sera à positionner dans le dossier de développement objet. Enfin, chaque **code** indique quelque chose à modifier dans le projet.

2.1 Lecture du code

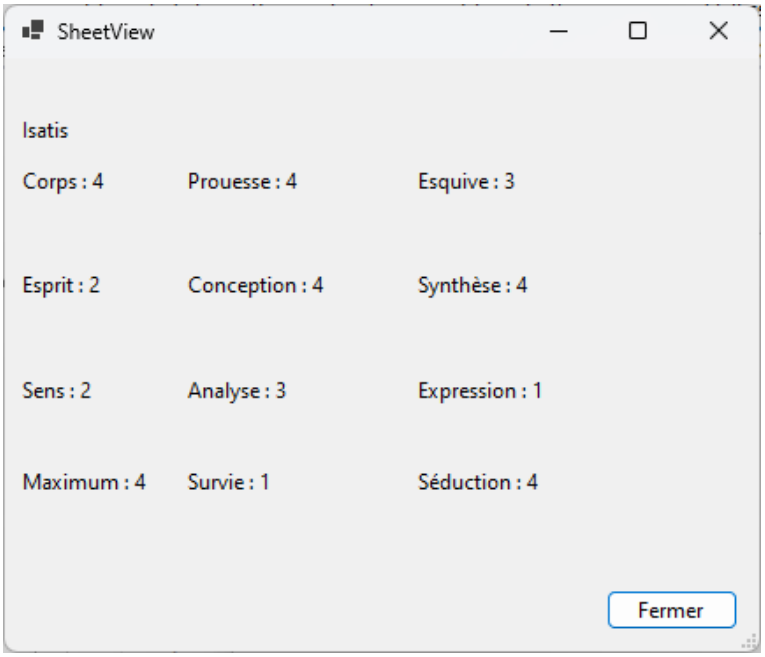
Commencer par essayer l'application telle que fournie. À l'ouverture, on dispose d'une fenêtre de *sélection* qui doit lister les personnages ; pour l'instant, elle dispose d'un bouton avec un personnage de nom aléatoire (en haut à gauche), d'un bouton pour créer un nouveau personnage (en haut à droite), d'un bouton pour quitter (en bas à droite). Ce dernier est fonctionnel. Le bouton en haut à droite semble ne rien faire, ou faire planter l'application si on clique dessus suffisamment, ce sera à modifier (bientôt). Le bouton avec un nom aléatoire (en haut à gauche), lui, ouvre une autre fenêtre.

Capture d'écran (page suivante) :



La fenêtre ouverte par le bouton en haut à gauche permet de visualiser une feuille de personnage – mystère. On visualise juste son nom et des données avec du texte suivi d'un chiffre ; il y a aussi un bouton pour fermer, qui fonctionne.

Capture d'écran :



Pour l'instant, c'est tout. Commencez par lire le code de `CharacterSelect` et `SheetView`, puis voyez les éléments auxquels ces fenêtres (`Form`) font référence.

2.2 Analyse du code

Exercice 1 Question : comment sont stockés les données numériques, indicateurs et compétences, pour chaque personnage décrit dans une `MysterySheet` ?

Exercice 2 Dossier : préparez le diagramme de classes UML **détaillé** intégrant les classes suivantes uniquement : `Roster`, `MysterySheet`, et les énumérations suivantes : `Indicator`, `Skill`, rien d'autre. Le rendu sera à effectuer dans le dossier de développement objets, voir consignes détaillées en Section 6, p. 23.

Exercice 3 Question : comment sont affichées les indicateurs et compétences dans une fenêtre `SheetView` ?

2.3 Premières modifications

Un premier problème évident sur la fenêtre `CharacterSelect` qui s'affiche à l'ouverture : elle a toujours `Form1` dans sa barre de titre... On préférerait autre chose de pertinent, comme `Sélection`.

Les autres fenêtres aussi portent le nom de leur classe.

Exercice 4 Code : corriger ces problèmes d'affichage.

Un premier objectif de modification est de remplacer le bouton par un affichage plus personnel (réalisé avec des méthodes `Draw`) pour les personnages. Dans la classe `CharacterSelect`, beaucoup de code non utilisé pour l'instant est déjà prêt à cet effet.

Ce code sera utilisé dans la suite du tutoriel. Pour l'instant, lisez la méthode `CreateCharacterViews` : elle fait appel à la classe `CharacterView`.

Exercice 5 Code : ajoutez après la dernière ligne du constructeur de `CharacterSelect` un appel à `CreateCharacterViews`.

Lisez le code des classes `CharacterView` et `ViewSettings`.

Exercice 6 Question : comment changer la couleur d'affichage des personnages ?

Pour réaliser l'affichage, du code doit afficher tous les personnages dans la réponse à l'événement `Paint`

Exercice 7 Question : comment sont stockées les vues des personnages affichées dans la classe `CharacterSelect` ?

Exercice 8 Code : dans la réponse à l'événement `Paint`, parcourez toutes les `views` pour les afficher.

Essayez : le personnage de base doit maintenant s'afficher (sous le bouton).

Il s'agit ensuite de pouvoir rendre ces vues cliquables : vérifiez le code qui permet d'ouvrir la fenêtre `SheetView`, qui est pour l'instant présent dans la méthode `CharacterButton1_Click`.

Exercice 9 Code : dans la réponse à l'événement `MouseDown`, parcourez toutes les `views`. Si le pointeur de la souris est contenu dans l'une d'entre elles, créer et afficher la `SheetView` correspondante.

Quant tout est fonctionnel, on peut maintenant se passer du bouton.

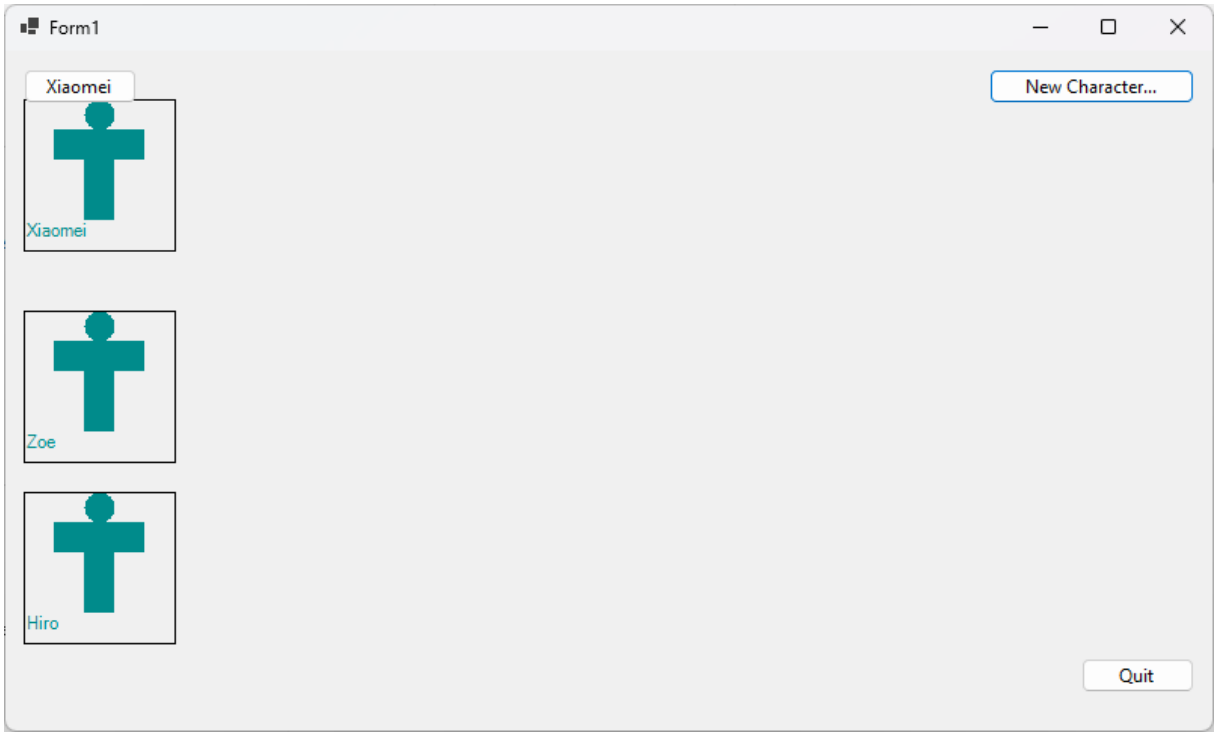
Exercice 10 Code : supprimer le bouton `characterButton1` et tout code qui y fait référence.

2.4 Batterie de feuilles

On s'intéresse ensuite à la création d'autres personnages. Pour l'instant, le bouton concerné permet de créer de nouveaux personnages, mais ne les affiche pas.

Exercice 11 Code : Dans la méthode `CreateRandomWithName`, ajoutez à la fin un appel à la méthode qui ajoute une nouvelle vue. Vérifiez que tout fonctionne.

On pourrait arriver à cette capture d'écran :



Vous devriez même avoir mieux, avec un titre à cette fenêtre et une interface en français, et un bouton redondant en moins, d'ailleurs.

Deux problèmes apparaissent, visibles si vous créez plusieurs personnages : un problème d'affichage, et un problème de plantage de l'application.

Exercice 12 Question : quels sont ces deux problèmes ?

L'affichage sera corrigé plus tard, mais on ne devrait pas avoir de plantage dans l'application.

Exercice 13 Code : corrigez la méthode `CreateRandomWithName` pour qu'elle ne fasse rien et affiche un message d'erreur (donné en commentaire) si le problème devait survenir.

2.5 Un premier dialogue

Plutôt que de tirer un nom au hasard, on veut pouvoir nommer les personnages. Dans la méthode de réponse au clic sur le bouton `createButton`, commentez la première ligne, décommentez la deuxième : elle fait appel à un `NewCharacterDialog`, examinez cette classe. Le bouton `Ok` ne fonctionne pas...

Exercice 14 Code : corrigez le fonctionnement du `NewCharacterDialog`.

Exercice 15 Dossier : donnez un diagramme de classes UML **général** du projet avec les types (classes, enum) suivants :

— Roster, MysterySheet, Indicator, Skill (comme précédemment, côté logique),

— CharacterSelect, CharacterView, SheetView (côté graphique).

Assurez-vous que tout est compris dans ce code...

2.6 Tests basiques

On veut pouvoir tester certains éléments. Le projet NUnit associé donne une ossature pour se faire. Voyez `BasicTests` pour quelques exemples d'utilisation de cette API de tests...

Essayez ce projet, il peut être nécessaire de nettoyer/régénérer le projet voire la solution avant de pouvoir exécuter les tests. Et parfois de relancer l'IDE.

Exercice 16 Code : Complétez la méthode `Skill_Tests` en vous inspirant des tests donnés dans `Indicator_Tests`. Instructions :

1. Créez une variable contenant la compétence de nom *Prouesse*,
2. Vérifiez que cette variable est non-nulle, que son nom est bien *Prouesse*, que son indicateur est bien *Corps*,
3. Vérifiez que la méthode `SkillPointsAtLevel` donne un résultat identique à son argument pour deux ou trois valeurs numériques.

Dans la classe `Trial`, on simulera des jets de dés. Pour l'instant, seule la méthode `NumericalResult` est donnée : cette méthode prend en paramètre une liste de nombres (les résultats de plusieurs dés différents) et renvoie leur somme, si la compétence utilisée à un niveau supérieur à 0. Voici les différents cas d'usage de cette méthode :

1. Cas 1 : la liste passée en paramètre est nulle, la méthode doit renvoyer la valeur 0.
2. Cas 2 : la liste passée en paramètre est vide, la méthode doit renvoyer la valeur 0.
3. Cas 3 : le niveau de compétence passé en paramètre est de 0, la méthode doit renvoyer la valeur 0.
4. Cas 4, cas nominal : la liste contient des nombres, le niveau de compétence est supérieur à 0, la méthode doit renvoyer la somme de la liste.

La méthode est correctement codée (à ce stade). Dans `TrialsTest`, une méthode auxiliaire déjà codée permet de tester correctement n'importe lequel de ces cas.

Exercice 17 Code : dans la méthode `NumericalResult_Test`, donnez un jeu de tests complet permettant d'appeler `NumericalResult_TestMethod` dans tous les cas d'usages possibles.

L'ensemble des tests doit passer.

2.7 Remise du tutoriel

Concertez-vous entre membres de l'équipe. Remettez votre code final avec tests et vos réponses aux questions : un rendu par équipe.

Ce tutoriel est terminé, passez à la suite...

Chapitre 3

Développement guidé

L'objectif est de passer du stade 1 (tutoriel terminé) au stade 2 (fonctionnalités de base réalisées). Dans ce chapitre, vous aurez des instructions assez directives mais moins détaillées que dans le tutoriel ; les objectifs restent assez simples et devraient être atteints rapidement. Organisez-vous au sein de votre équipe pour que chaque personne maîtrise cette partie. Ici, il n'est question que de **code**.

3.1 Édition basique

En plus de voir un personnage créé aléatoirement, on souhaite pouvoir *éditer* les valeurs pour terminer la création. Dans l'idée :

1. Pour chacun des trois indicateurs (Corps, Esprit, Sens), on peut retirer un niveau si l'indicateur est strictement supérieur à 1, ce qui donne des points d'indicateur ; on peut rajouter un niveau si ce niveau n'est pas déjà au seuil maximum de création et qu'on a les points d'indicateur suffisants,
2. L'indicateur `Maximum` est le maximum des trois autres,
3. Pour chaque compétence, on peut retirer un niveau si ce dernier n'est pas 0, ce qui donne des points de compétence ; on peut rajouter un niveau si ce niveau n'est pas déjà au seuil maximum de création et qu'on a les points de compétence requis.

À votre disposition pour mettre ceci en place :

1. Une barre d'outils, à rendre `Visible`,
2. Un dialogue d'édition, `SimpleEditDialog`,
3. Du code présent dans la classe `SheetView`

Exercice 18 Code : analysez les éléments fournis et rendez l'édition fonctionnelle.

3.2 Génération fixe

La règle de génération totalement aléatoire utilisée est problématique, pouvant donner des résultats totalement aberrants. On propose, en première alternative, une création à points fixes : tous les indicateurs et compétences ont un même niveau de base (on utilise ensuite l'édition pour modifier ces valeurs).

Exercice 19 code : Voyez l'énumération `Rules`, mettez en place un cas pour `Rules.Fixed` dans la méthode `RandomGeneration` qui utilise les valeurs `BasicValue`. Ensuite, modifiez `NewCharacterDialog` en ajoutant l'option d'utiliser ces règles. Ajoutez des tests unitaires pour valider ces résultats.

3.3 Correction d'affichage

Pour l'instant, si on crée de nombreux personnages, ils dépassent du cadre de la fenêtre `CharacterSelect`, il faudrait remédier à ce problème d'affichage.

Exercice 20 Code : modifiez l'affichage effectué dans les méthodes `CreateCharacterViews`, `AddOneCharacterView` pour revenir en haut de la fenêtre (avec un décalage horizontal) quand la vue suivante sortirait du cadre. Évaluez le nombre de personnages maximum qui peut être créé et désactivez le bouton de création quand ce seuil est atteint.

Gérer cet ensemble de personnages suppose aussi de pouvoir en supprimer.

Exercice 21 Code : ajoutez une fonctionnalité (bouton, menu, clic droit...) permettant de supprimer un personnage, après validation à l'aide d'un dialogue de confirmation (*Êtes-vous sûr ?*.

3.4 Ajout : niveaux de compétence

Les indicateurs sont numériques, les compétences devraient refléter des *niveaux de compétence*. L'objectif est qu'à la place d'un nombre, on affiche la compétence avec une description textuelle. Voici la correspondance de ces niveaux :

Niveau	Description
0	Incompétent
1	Amateur
2	Apprenti
3	Compagnon
4	Maître
5	Agrégé
6	Ministre

Exercice 22 Code : Ajoutez aux `Skill` les fonctionnalités nécessaires pour faire correspondre chacun de ces chiffres de 0 à 6 au texte associé. Utilisez ce code pour l'affichage dans la `SheetView`.

3.5 Épreuves et difficulté

L'objectif de l'ensemble des fonctionnalités qui suivent est de permettre à un personnage de passer une *épreuve* : d'utiliser ses compétences, indicateurs et quelques dés pour tenter d'y répondre. Il y a deux types d'épreuves : les *épreuves ouvertes*, comme le concours de saut en longueur par exemple, où on cherche simplement à savoir ce que peut faire le personnage, et les *épreuves fermées*, comme le fait de sauter par-dessus un précipice d'une longueur donnée dans laquelle on donne une difficulté et on cherche à savoir si le personnage réussit on non. (Les conséquences de rater un saut au-dessus d'un précipice ne sont pas encore gérées ici.)

Pour cela, on dispose d'une échelle de conversion de nombres en difficulté. Pour réussir à une certaine difficulté, il faut au moins atteindre le score correspondant.

Voici cette échelle :

Difficulté	**Score (minimum)**
Rien	0
Trivial	5
Facile	10
Normal	15
Complexe	20
Délicat	25
Défi	30
Exploit	50
Surhumain	70
Impossible	100

Exercice 23 Code : Créez une énumération `Difficulty`, le code nécessaire pour avoir la correspondance entre ses éléments, le texte et le score approprié, et une méthode qui pour un score (nombre entier) donné, renvoie la difficulté maximum atteinte avec ce score. Validez le bon comportement de cette méthode par des tests unitaires.

Exercice 24 Code : Dans la classe `Trial`, créer une méthode `Cast`, prenant en paramètre un nombre total de dés à lancer et renvoyant une liste d'entiers (tirage aléatoire entre 1 et 6) de cette longueur correspondant à ce tirage. Validez le bon comportement de cette méthode par des tests unitaires.

3.6 Résultats des épreuves, version simple et tests

Pour les épreuves ouvertes, on donne un personnage, un indicateur à utiliser et une compétence. On lance un nombre de dés correspondant au niveau de l'indicateur, et on utilise le niveau (numérique) de la compétence pour donner le résultat à l'aide de la méthode `NumericResult` ; ce résultat est ensuite converti en difficulté atteinte. Pour les épreuves fermées, on donne la difficulté à atteindre ; si elle est atteinte, on renvoie `true`, sinon, `false`.

Exercice 25 Code : coder deux méthodes dans la classe `Trial` et les tests unitaires complets associés, correspondant aux épreuves ouvertes et fermées.

3.7 Interface d'épreuve

Il s'agit ensuite d'ajouter une interface permettant de sélectionner un personnage et une compétence, et de lancer une épreuve, puis d'afficher un résultat.

Exercice 26 Code : créez d'abord une nouvelle fenêtre associée à une `MysterySheet`. Cette fenêtre affiche le nom du personnage, celui de la compétence visée, l'indicateur correspondant, les niveaux associés, et soit la difficulté ciblée, soit les mots *épreuve ouverte*. Un bouton permet de lancer l'épreuve (une seule fois, ce bouton est désactivé ensuite) ; le résultat doit s'afficher. Un autre bouton ferme cette fenêtre.

En attendant l'exercice suivant, vous pouvez poser dans `CharacterSelect` un bouton lançant cette fenêtre avec des valeurs fixes.

3.8 Ergonomie : glisser-déposer

Il est de bon ton d'avoir une interface visuelle agréable.

Exercice 27 Code : dans `CharacterSelect`, ajoutez du code pour délimiter quelques zones à afficher dans la partie droite de la fenêtre : ces zones vont délimiter une *Salle d'auto-évaluation* (pour les épreuves ouvertes), une *Salle d'examen* (pour les épreuves fermées), une *Salle d'entraînement* (gérée plus tard). Ajoutez aussi une `ComboBox` permettant de choisir la compétence ciblée, ainsi qu'une pour la difficulté de l'épreuve fermée.

Exercice 28 Code : faites en sorte qu'on puisse *glisser-déposer* les personnages dans ces zones. Pour les deux premières, cela déclenche respectivement une épreuve ouverte ou fermée.

3.9 Récapitulatif

Après avoir réalisé l'ensemble de ces ajouts dans le code, vous disposez désormais d'une application au Stade 2 proposant de multiples fonctionnalités et s'appuyant sur des tests solides.

Chapitre 4

Développement décrit

L'objectif est de passer du stade 2 (fonctionnalités de base) au stade 3 (application complète). Dans ce chapitre, vous aurez des instructions plus générales que dans le chapitre précédent, et il est attendu une certaine autonomie dans l'interprétation des consignes et dans votre organisation. Il est aussi possible pour vous de choisir les fonctionnalités que vous implémentez, si vous n'avez pas le temps de tout faire.

4.1 Cohérence des données

Exercice 29 Code : assurez-vous (si ce n'est pas encore fait) que l'édition de l'indicateur `Maximum` est impossible – celui-ci doit toujours être le maximum des trois autres.

4.2 Correction d'ergonomie

Exercice 30 Code : proposez un meilleur dialogue d'édition, qui donne à l'utilisateur la possibilité d'ajouter ou retrancher plusieurs points d'un coup et de revenir en arrière, en indiquant aussi les coûts et gains de chaque opération.

4.3 Configurer quelques variables

Exercice 31 Code : créez un dialogue de préférences permettant de configurer quelques éléments : couleur de fond, couleur des personnages... ainsi que le nombre de personnages à créer au démarrage. Ouvrez ce dialogue avant le lancement de la fenêtre `CharacterSelect`.

4.4 Enchaîner les épreuves

Exercice 32 Code : Ajoutez un `NumericUpDown` ou élément équivalent pour choisir le nombre d'épreuves à lancer. Quand elles sont lancées, affichez une boîte de dialogue avec l'enchaînement de tous les résultats.

4.5 Entraînement et expérience

Un personnage peut être dans deux états : *création* et *formation*. Quand il est en phase de création, on peut modifier ses indicateurs et compétences avec les fonctionnalités vues précédemment. Quand il est en phase de formation, ce n'est plus possible, mais il peut passer des épreuves.

Exercice 33 Code : ajoutez le code nécessaire, y compris un bouton pour valider le personnage (passant définitivement de *création* à *formation* le blocage des *épreuves* pour un personnage en *création*, le blocage de l'édition pour un personnage en *formation*.

Un personnage en formation gagne de *l'expérience* : un point d'expérience à chaque fois qu'il réussit une épreuve fermée.

Exercice 34 Code : ajoutez cette donnée, sa visualisation sur `SheetView`, et son ajout dans les épreuves fermées.

Les points d'expérience servent à *entraîner* le personnage. En allant dans la *salle d'entraînement* avec suffisamment de points d'expérience, il peut gagner des points dans une compétence ou un indicateur ; le nombre de points d'expérience nécessaire est celui en points d'indicateur ou en points de compétence nécessaires pour le niveau suivant. Le niveau d'une compétence ou d'un indicateur peut dépasser le seuil de création, mais pas le maximum prévu.

Exercice 35 Code : créez une interface pour la salle d'entraînement, rendez-la fonctionnelle.

4.6 Plans, sphères et compétences

Chaque personnage, à sa création, est inscrit dans un *Plan* et étudie une *Sphère*. Ces éléments modifient la liste des compétences disponibles, voyez le descriptif complet en annexe – B.3, p. 36 et B.3, p. 36.

Exercice 36 Code : concevez et ajoutez les classes, énumérations et éléments nécessaires au code existant. Modifiez le dialogue de création de personnage pour sélectionner le plan et la sphère. Affichez les résultantes.

4.7 Épreuves et compétences : compétences absentes

Avec ces nouvelles compétences, il peut y avoir des personnages qui subissent des épreuves dans une compétence qu'ils ne peuvent pas avoir. Il peut aussi y avoir des personnages incompétents (ayant le niveau 0) dans une compétence. Dans cas, il est tout de même possible de passer cette épreuve :

— en choisissant une compétence du même indicateur,

— en retranchant deux niveaux à la difficulté obtenue.

Exercice 37 Code : modifiez le code des épreuves en conséquence.

4.8 Épreuves et compétences : modificateurs de total

Les niveaux de compétence ont des effets : le simple fait d'être compétent ajoute une valeur automatiquement au total obtenu par les dés. Voici ces modificateurs :

Compétence	Modificateur
Apprenti	+5
Compagnon	+5
Maître	+10
Agrégé	+15
Ministre	+20

Exercice 38 Code : modifiez le code des épreuves en conséquence.

4.9 Épreuves et compétences : résultats automatiques

Pour les épreuves fermées, dans certains cas, il n'y a pas lieu de lancer les dés, certains résultats sont automatiques. Les voici :

Compétence	Réussite automatique	Échec automatique
Incompétent	Aucune	Toutes
Amateur	Aucune	Exploit et supérieures
Apprenti	Trivial	Surhumain et supérieures
Compagnon	Trivial	Impossible
Maître	Facile et inférieures	Aucune
Agrégé	Normal et inférieures	Aucune
Ministre	Complexe et inférieures	Aucune

Exercice 39 Code : modifiez le code des épreuves en conséquence.

4.10 Épreuves et compétences : modificateurs de jet

Les compétences reflètent aussi la maîtrise, ce qui modifie la manière de comptabiliser les jets de dé. Voici comment :

Compétence	Chaque...	Compte pour...
Amateur	1	0
Compagnon	6	10
Maître	5 ou 6	10
Agrégé	4, 5, ou 6	10
Ministre	4, 5 ou 6	20

Exercice 40 Code : modifiez le code des épreuves en conséquence.

4.11 Épreuves et compétences : tests de la version complète

Avec l'ensemble des modifications précédentes, les épreuves deviennent plus complexes dans leur code et dans leur vérification.

Exercice 41 Dossier : en prenant en compte toutes les spécifications, préparez, pour la méthode permettant de savoir quelle difficulté a passée un personnage sur une épreuve, l'ensemble des cas d'usage possible en détaillant toutes les situations et les résultats attendus pour cette méthode. Donnez la liste complète de ces cas d'usage dans le dossier de conception – exercice à réaliser même si le code correspondant n'est pas fait.

Exercice 42 Code : réalisez les tests et jeux de tests complets correspondant. Assurez-vous de la conformité de votre code à ces tests.

4.12 Qualité de code, commentaires de documentation

Exercice 43 Code : il est maintenant conseillé de reprendre le code et de le refactoriser si nécessaire pour réduire les redondances et mauvaises pratiques, et vous assurer de sa qualité en matière de découpage en classes et méthodes, de nommage, de respect des normes. Ajoutez aussi la documentation de vos types, propriétés, constructeurs et méthodes publiques.

4.13 Récapitulatif

Après avoir réalisé autant que possible de ces ajouts dans le code, vous disposez désormais d'une application au Stade 3. Les derniers ajouts sont à réaliser si vous souhaitez aller plus loin, et si vous avez fini tout ce chapitre.

Chapitre 5

Développement induit

L'objectif est d'atteindre le stade 4+. Ce chapitre contient plusieurs fonctionnalités non prioritaires, à vous de choisir celles que vous préférez ainsi que la manière de les réaliser.

5.1 Processus de création

Au lieu de créer un personnage avec des points libres à répartir, on utilise le processus décrit en annexe – voir B.3, p. 38

Exercice 44 Code : ajoutez code et interface pour la création par étapes.

5.2 Données supplémentaires et accessoires

Chaque fiche comprend aussi d'autres types de données : richesse, âge (numérique), titre (texte), inventaire (liste de textes), relations (avec d'autres personnages)... voire un visuel.

Exercice 45 Code : modifiez la fiche et permettez l'édition (libre) de ces données supplémentaires.

5.3 Meilleurs visuels

Ce qu'il y a est très basique. N'hésitez pas à faire mieux.

Exercice 46 Code : utilisez vos compétences de graphisme, design et ergonomie pour améliorer ces aspects, en privilégiant toujours les fonctionnalités et l'expérience utilisateur.

5.4 Chargement des données depuis des fichiers

Deux parties ici :

Exercice 47 Code : permettez de sauvegarder un personnage (avec tout ce qu'il y a dedans, ce serait dommage de le perdre). Et bien sûr de le recharger depuis un fichier ensuite.

Exercice 48 Code : plutôt que d'avoir les données codées en dur (minimum = 0 ou 1, la liste des compétences, etc.), faites en sorte qu'elles soient chargées depuis un (ou plusieurs) fichiers décrivant les règles à suivre.

5.5 Points de talent

En formation, les Mystères passent parfois des épreuves pour lesquelles ils ne sont pas entraînés. Pour cela, ils disposent d'une ressource précieuse : le Talent.

Son utilisation est détaillée en Annexe B.3, p. 42.

Exercice 49 Code : ajoutez des points de talent (commençant à 10) aux personnages. On peut utiliser un point de talent pour (un seul usage possible par point) :

— faire passer la difficulté d'une épreuve fermée, si elle était entre Complexe et Défi, à Facile,
— ajouter 10 au résultat d'un jet de dés,
— relancer une fois les dés *après* une épreuve,
— ne pas faire un jet et obtenir Normal.

5.6 Santé mentale

Au fur et à mesure de leur formation, les Mystères sont sujet au doute, à la dépression, à l'anxiété, et à la folie pure... Cela dépend de leur santé mentale. Les points de santé mentale commencent à $10 \times m$, où m est le *minimum* des trois indicateurs. À chaque épreuve fermée échouée, les Mystères perdent un point de santé mentale. Ils peuvent le regagner à la deuxième épreuve réussie. Si la santé mentale arrive à 0, les points de talent ne peuvent plus être utilisés, et la difficulté est décalée de deux niveaux vers le bas.

La santé mentale est détaillée en Annexe B.3, p. 41.

Exercice 50 Code : faites.

5.7 Restructuration code, interface et qualité

Prévoyez quelques temps en fin de projet pour revenir sur les fichiers, leurs structures et reprendre le tout en termes de code, d'interfaces humain-machine et de qualité : tests, documentation, normes.

5.8 L'application terminée

Une fois que toutes les personnes de l'équipe sont parfaitement satisfaites, effectuez le rendu de l'archive contenant le résultat.

Exercice 51 Dossier. N'oubliez pas le diagramme de classes final et le dictionnaire de responsabilités à établir, voir Chapitre 6, p. 23.

Chapitre 6

Développement objets : le dossier

6.1 Consignes

On vous demande **un rendu unique** à effectuer sur Moodle en temps et en heure, sous la forme d'**un seul fichier PDF**.

Son nom devra être `NOM1_NOM2_NOM3_développement.pdf` avec les noms de famille de chaque membre de l'équipe.

On demande une page de garde (titre, noms de chaque personne dans l'équipe de nouveau, groupe de TD), puis une page par diagramme ou rendu. Chacun des rendus doit aussi avoir un titre.

Il n'y a pas besoin d'autres éléments (ni introduction ni conclusion dans ce document).

6.2 Les diagrammes initiaux

1. Exercice 2, partie Tutoriel : Un diagramme de classes UML détaillé, c'est-à-dire incluant les classes et énumérations, leur contenu avec noms, types, signatures, visibilités, les relations entre ces types. Ce diagramme inclus les types `Roster`, `MysterySheet`, `Indicator` et `Skill`, ainsi que les types associés si nécessaire.
2. Exercice 15, partie Tutoriel : Un diagramme de classes UML général, c'est-à-dire ne donnant qu'un rectangle avec le nom du type pour les classes et énumérations, mais qui donne tout de même les relations entre les éléments. Comprend les quatre types demandés dans le diagramme précédent, qui font partie de la logique métier, plus également les classes `CharacterSelect`, `CharacterView` et `SheetView`, qui font partie de l'interface humain-machine ; ne détaillez pas le découpage en paquets ici, juste les relations entre types.

6.3 Le diagramme final

On demande enfin, Exercice 51 **quel que soit votre état d'avancement**, un diagramme final du projet. Votre application peut en être au stade 1, 2, 3, 4 ou plus : donnez le diagramme de classes général (sans détailler le contenu des types donc) de tout le projet. Vous pouvez omettre les classes *qui ne sont associées à aucune autre* et ne font que des *usages* d'autres classes comme `Program`, par exemple, tout le reste doit y figurer. Ce diagramme sera bien sûr différent d'une équipe à l'autre.

Attention, le diagramme final peut être complexe. Assurez-vous qu'il soit lisible...

6.4 Le dictionnaire des responsabilités

En plus du diagramme final, on demande, sur une page, un *dictionnaire de responsabilités* : pour chaque type (classe, interface, énumération...) présent dans le diagramme final, écrivez quelques mots (deux lignes maximum, probablement moins) expliquant *ce que fait* ou *à quoi sert* chacun des types.

6.5 Qualité : code, tests, documentation

Il est demandé explicitement en Exercice 41, et même si vous n'avez pas effectué le code des exercices précédents, une analyse complète des cas d'usage de la méthode permettant d'obtenir la difficulté passée par le personnage sélectionné, étant donné une série de nombre représentant les jets de dés, ainsi que le niveau de la compétence utilisée, avec l'ensemble des règles.

Cette analyse des cas d'usage de la méthode peut être donnée sous forme de texte, tableau(x), ou les deux.

La qualité du code, le nommage, le découpage, les tests en eux-mêmes et la documentation font partie de votre rendu de code. Ces éléments ne doivent pas figurer dans le dossier PDF rendu.

Chapitre 7

Le compte-rendu en deux parties

7.1 Consignes

On vous demande **deux rendus** à effectuer sur Moodle en temps et en heure, sous la forme d'**un fichier PDF chacun**.

7.2 Compte-rendu commun

Le premier compte-rendu est **à rédiger en commun**. Son nom devra être `NOM1_NOM2_NOM3_compteRendu.pdf` avec les noms de famille de chaque membre de l'équipe. Précisez simplement en une à trois pages la liste de tous les exercices que vous avez :

— complètement terminés,

— effectués partiellement, en indiquant ce qui est fait dedans en quelques mots, et votre interprétation des consignes s'il y a lieu.

Ce compte-rendu peut être fait sous la forme d'un tableau. Ne précisez pas ce qui n'est pas fait, il n'y a pas besoin d'introduction ou de conclusion.

7.3 Compte-rendu individuel

Le second compte-rendu est **à rédiger individuellement**, et à rendre individuellement. Son nom devra être `NOM_personnel.pdf` avec votre nom. L'objectif est de répartir $12 \times n$ où n est le nombre de personnes dans l'équipe points, en expliquant brièvement pourquoi vous donnez chaque note à qui – si tout le monde a également contribué, mettez 12 à tout le monde. Vous êtes inclus dans cette liste.

Ce ne sera pas *directement* votre note d'évaluations par les pairs, l'enseignant utilisera ceci et votre attitude pendant la SAÉ pour l'attribuer – la moyenne ne sera pas 12 mais votre vraie note de fonctionnalités.

Chapitre 8

Récapitulatif : livrables, dates, évaluations

Ceci est un récapitulatif aussi complet et direct que possible.

En cas d'impondérables : maladie, indisponibilité longue d'une, défection d'un membre de l'équipe... Prévenez immédiatement **vos** enseignants (chargés du suivi en R2.01 et R2.02), **avant** la date limite.

Bonne SAÉ !

— Livrables :

1. Rendu tutoriel - projet : en .zip (ou .7z), sur Moodle, date 1, évaluation 1.
2. Rendu des questions posées dans le tutoriel : en .txt, sur Moodle, date 1, évaluation 1.
3. Rendu du projet fini : en .zip (ou .7z), sur Moodle, date 2, évaluations 2, 3, 6.
4. Rendu du dossier de développement objets : en PDF, sur Moodle, date 2, évaluation 5.
5. Rendu compte-rendu commun : en PDF, sur Moodle, date 2, nécessaire mais non évalué, servant à l'évaluation 4.
6. Rendu compte-rendu individuel : en PDF, sur Moodle, un par personne, date 2, nécessaire, servant à l'évaluation 4 aussi.

— Dates :

1. Vendredi 3 avril 2026, 18:00 (tutoriel et questions).
2. Lundi 27 avril 2026, 8:00 (tout le reste).

— Évaluations (somme des coefficients : 100) :

1. Évaluation questions et tutoriel, coefficient 2 (R2.02)
2. Évaluation fonctionnalités et ergonomie, coefficient 11 (R2.02)
3. Évaluation qualité, découpage, etc. coefficient 7 (R2.02)
4. Évaluation par les pairs (modérée par l'enseignant) coefficient 7 (R2.02)
5. Évaluation du dossier de développement objets, coefficient 12 (R2.01)
6. Évaluation des tests (code) coefficient 8 (R2.01)
7. Évaluation en DS de SAÉ coefficient 26 (R2.02)
8. Évaluation en DS de SAÉ coefficient 15 (R2.01)
9. Report de la note du DS 1 R2.01 coefficient 6
10. Report de la note du DS R2.02 coefficient 6

Annexe A

C# : rappels et compléments

A.1 Visibilités en C# et UML

`public`, `private` ont le même sens qu'en Java.

La visibilité `internal` correspond exactement à la visibilité de paquet en Java ; elle se note ~ en UML.

Attention, les visibilités par défaut (sans mot-clé) sont différentes :

— dans une classe (attributs, propriétés, constructeurs, méthodes) : la visibilité par défaut est `private`,

— qualifiant une classe : la visibilité par défaut est `internal`,

— dans une interface : la visibilité par défaut est `public`.

Trois variantes de `protected` existent en C#... Celle correspondante à celle de Java est `protected public`.

A.2 Propriétés en C# et en UML

Les propriétés en C# sont une notation pratique et très utilisée pour agréger des notions diverses : attributs, accesseurs, méthodes... Notions qui existent en UML (et en Java), mais séparément. Il y a plusieurs cas d'utilisation :

1. La propriété calculée :

```
public Point Centre { get { return new(Location.X+Size.Width/2,
                        Location.Y+Size.Height/2); } }
```

   est équivalente à une méthode, en UML :

   `+Centre.get(): Point`

2. La constante :

```
public int Max { get; } = 10;
```

   est équivalente à :

   `+Max: int=10`

3. Une propriété complète (accès et modification) synthétise trois éléments : un attribut (non accessible directement), son accesseur – méthode `get`, son mutateur – méthode `set`, qui ont la visibilité prévue. Par exemples, quelques entiers :

```
public int A { get; set; }
public int B { get; internal set; }
internal int C { get; private set; }
private int D { get; set; }
```

   se traduiront en UML par :

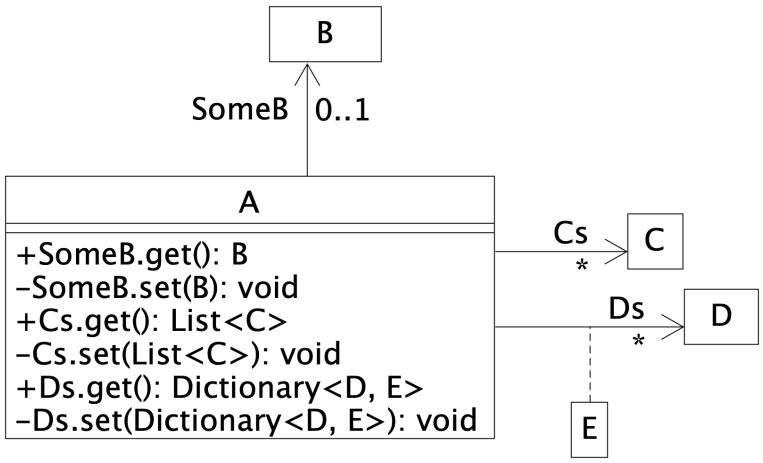
```
-A: int
-B: int
-C: int
-D: int
------
+A.get(): int
+A.set(int): void
+B.get(): int
~B.set(int): void
~C.get(): int
-C.set(int): void
-D.get(): int
-D.set(int): void
```

4. Ne pas tenir compte dans le diagramme UML des éléments notés `init` (utiles uniquement à l'initialisation).

5. Une propriété dénotant une association doit faire apparaître ses `get` et/ou `set` en tant que méthodes, mais l'attribut n'est pas dans la liste des attributs puisque c'est une association :

```
public class A {
  public B SomeB { get; private set; }
  public List<C> Cs { get; private set; }
  public Dictionary<D, E> Ds {get; private set; }
  //...
}
```

   donnera lieu au diagramme de classes (détaillé pour A uniquement) suivant (page suivante) :



A.3 Valeur par défaut des paramètres

Dans une méthode, en spécifiant les paramètres, on peut donner une valeur par défaut à certains. La contrainte est que ces paramètres *optionnels*, qui ont donc (chacun) une valeur par défaut, soient après les paramètres *obligatoires*, qui doivent être fournis. Syntaxe :

```
public X Y(type1 a1, type2 a2, ..., typeO1 o1=v1, typeO2 o2=v2, ...)
```

En pratique, si on appelle la méthode, on peut ou non donner les arguments o – si ce n'est pas le cas, les valeurs v seront utilisées à la place. Il est recommandé d'avoir au plus un paramètre avec valeur par défaut. Ce mécanisme existe aussi en Java, Python et dans de nombreux autres langages.

A.4 Opérateur ternaire

L'opérateur `?:` permet d'écrire en une ligne une expression dépendant d'une condition booléenne : `+c?a:b` vaut `a` si `c` est vraie et `b` si `c` est fausse. Cette opération simple vient du langage C, et existe également en C++ et Java (entres autres langages). Il est recommandé de ne pas enchaîner plusieurs de ces constructions, qui rendent rapidement la lecture du code difficile.

A.5 Abréviations

Les méthodes simples comme :

```
public X Y() {
  return Z;
}
```

...sont souvent abrégées en : `public X Y() => Z;` – rien ne change, mais l'écriture est faite sur une ligne.

A.6 Pré-appels dans les constructeurs

Il est régulièrement nécessaire d'appeler le constructeur de la super-classe dans les classes spécialisées. En Java, le code est :

```
public MyExtendedClass(Type argument) {
  super(argument);
  //...
}
```

En C#, la structure est un peu différente et utilise le mot-clé `base` :

```
public MyExtendedClass(Type argument): base(argument) {
  //...
}
```

De même, quand le constructeur est surchargé (défini avec plusieurs signatures), il est courant d'appeler le constructeur le plus élémentaire dans le plus complexe. En Java :

```
public MyClass() {
  //... (simple constructor code)
}
public MyClass(Type argument) {
  this(); // calls the simple constructor first
  attribute = argument; // then specific code
}
```

La structure C# est proche de la précédente :

```
public MyClass() {
  //... (simple constructor code)
}
public MyClass(Type argument): this() {
  attribute = argument;
}
```

A.7 Tuples

C# permet d'utiliser des `tuples`, connus en Python, avec la notation `(a, b, c, ...)` : éléments du tuple entre parenthèses séparés par des virgules. On peut ainsi écrire l'échange de deux variables a et b avec des couples : `(a, b)=(b, a);`..

A.8 Les enum en C#

Comme en Java, les énumérations ou **types énumérés** sont déclarées comme des classes en remplaçant le mot-clé `class` par `enum`. Les éléments d'un type énuméré ont un nom commençant par une majuscule (mais pas tout en majuscule), comme les constantes. Contrairement à Java, les types énumérés ne peuvent pas contenir de méthodes ou constructeurs, juste la liste des valeurs qu'ils peuvent prendre ; une variable déclarée avec un type énuméré a forcément une valeur, `null` ne peut pas être utilisé. Il arrive fréquemment de nuller une variable de ce type, et/ou d'utiliser une valeur spécifique `None`. Les types énumérés sont fréquemment accompagnés de `classes-compagnes` qui donnent des méthodes utilitaires agissant avec ou sur ces valeurs.

A.9 Méthodes d'extension

Une méthode d'extension est une méthode de classe qui permet d'ajouter une méthode d'instance à un autre type que la classe courante, sans nécessiter d'hériter de dernier type (ce qui est parfois impossible). **Attention**, ne l'utilisez pas pour quelque chose qui peut être réalisé normalement par les mécanismes classiques (une classe que vous définissez doit être donnée avec toutes ses méthodes) ; ce mécanisme n'est utile que pour suppléer une lacune de l'API ou un blocage due au langage (par exemple, donner une méthode utilisable sur un type énuméré, ce qui n'est pas directement possible (voir précédemment)).

Le premier argument est décoré par le mot-clé `this`. Par exemple, le code suivant permet d'ajouter une méthode au type `bool` :

```
public static class BoolAddition {
  public static string WinningResult(this bool x) {
    if (x) {
      return "You won!";
    }
    return "You lost...";
  }
}
```

Si la classe précédente est accessible, le code suivant pourra être utilisé :

```
bool b = GameResult();
resultLabel.Text = b.WinningResult();
```

A.10 is, as

Quand un type est incertain, l'opérateur `is` permet de le vérifier et de le convertir, et `as` permet de le convertir directement.

Par exemple, pour un événement émis par un bouton :

```
private void ButtonX_MouseDown(object sender, MouseEventArgs e) {
  Button button = sender as Button;
  ...
}
```

L'opérateur `as` effectue la conversion, si cette dernière est impossible, la référence `null` est renvoyée (sans lever d'exception).

Pour un parcours des contrôles d'une fenêtre, vérification avec conversion :

```
foreach (Control c in Controls) {
  if (c is Label label) {
    ...
  }
  ...
}
```

`is` permet à la fois de vérifier le type et de créer une variable locale du type le plus précis.

Attentions, ces deux opérateurs ne peuvent être invoqués que sur des types nullables.

Annexe B

Les Mystères de BUTopia

B.1 BUTopia

Je n'oublierai jamais les vies passées à BUTopia. Enfants, nous jouions au milieu de labyrinthes de meubles, désordonnés, immenses, chargés de volumes poussiéreux, porteurs de savoirs inconnus ; nous courrions entre les immeubles, dans les allées des grandes salles de cours, entre les majestueuses tours d'ivoire des maîtres ; aînés, nous clopinions péniblement entre les bibliothèques, musées et archives, usant nos bésicles sur des textes obscurs, des parchemins chargés de symboles, des diagrammes donnant le tournis mais porteurs d'un sens profond, laissant entrevoir la transcendance d'un absolu insaisissable, qui semblait perpétuellement rester tout juste hors de notre portée.

Combien d'années, de siècles, de vies avons-nous consacré à ce sacerdoce ? Les origines se perdent dans mon esprit, mais je sais qu'il nous faut poursuivre, nous Mystères, car nous sommes capables de manier les Sphères de la connaissance et de composer la parole qui, couchée par écrit, devient le Code, et ce Code réifie notre volonté. Nous ne sommes pas comme les autres, car nous sommes capables de Développer. Et si nous sommes venus à BUTopia étudier auprès des maîtres, en suivant le programme qui leur a été révélé par la très sainte DGESIP et ses Ministres successifs, c'est pour optimiser sans cesse nos Indicateurs clés de performance et travailler de manière critériée à la réussite de nos Compétences dans la Quête sempiternelle de l'amélioration continue proactive qui nous mènera un jour, nous en avons la foi, au BUT.

BUTopia est un univers fictif, fantastique et nous prétendrons qu'il n'a aucun rapport avec. Il est centré sur l'apprentissage mystique, la quête initiatique suivie par de nombreuses personnes évoluant dans ce territoire, tendant vers un BUT ; plusieurs problématiques peuvent donner lieu à plusieurs applications ou jeux différents.

B.2 Les Mystères

Ici, nous sommes centrés sur les Mystères de BUTopia, c'est-à-dire les personnes fictives – *personnages* – qui y suivent un apprentissage. *Les Mystères de BUTopia* est un jeu de rôles fictif – vous pouvez l'approcher comme un jeu de rôles sur table ou un jeu vidéo, ou simplement comme le répertoire de personnages d'une fiction qui n'est pas complètement écrite [1].

Le système détaillé ci-après récapitule les éléments de règles utilisés.

B.3 Détails du système

Les Plans

Tout Mystère suit un Plan : un archétype de type de formation, qui donne un *adjectif* qualifiant le personnage. Les voici :

Nom	Indic	C. Corps	C. Esprit	C. Sens	C. Max
Penseurs	Esprit	Concentration	Abstraction	Visualisation	Modélisation
Légats	Sens	Pérennité	Perception	Détails	Mémoïsation
Faiseurs	Corps	Habileté	Architecture	Organisation	Réification
Intégrateurs	Max	Proprioception	Liaison	Synesthésie	Compilation
Commerçants	Sens	Endurance	Finance	Expertise	Communication

Chaque Plan apporte une compétence au Corps, à l'Esprit, au Sens, et au Maximum.

Les Sphères

Tout Mystère étudie une Sphère : un domaine théorique et pratique qui permet d'agir. Les voici :

Nom	Indic	C. Corps	C. Esprit	C. Sens	C. Max
Algèbre	Esprit	Jonction	Induction	Équation	Fonctionnel
Verbe	Sens	Injonction	Formulation	Poétique	Objet
Transition	Max	Déconstruction	Anticipation	Argumentation	Optimisation
Création	Corps	Ajustement	Conception	Narration	Données
Valorisation	Max	Destruction	Intendance	Conviction	Tests

Chaque Sphère apporte aussi une compétence au Corps, à l'Esprit, au Sens, et au Maximum.

1. Au cas où : auteur = Bruno Mery, 2026, tous droits réservés. Merci de me contacter si vous utilisez le concept pour un MMORPG ou un roman à succès qui vous rend millionnaires.

Les Indicateurs

Les Indicateurs clés de performance (ou indicateurs) donnent un niveau numérique aux capacités de chaque personne en ce qui concerne trois aspects majeurs de leur être (fictif toujours).

1. Le Corps : une mesure des capacités physiques incluant la robustesse, la souplesse, la puissance, la dextérité, l'agilité, la rapidité...
2. L'Esprit : une mesure des capacités intellectuelles dont le raisonnement, l'imagination, l'abstraction...
3. Les Sens : une mesure de la capacité d'interagir avec son environnement et autrui, que ce soit vers soi : regarder, écouter, comprendre, ou vers les autres : parler, écrire, projeter ses émotions...
4. Le Maximum : il s'agit du maximum de ces trois mesures, utilisé pour toutes les compétences qui peuvent faire appel à l'une d'entre elles suivant la préférence de la personne qui les met en œuvre. Pour la survie en milieu hostile : forêt, désert, ou jury d'examen, une personne emploiera ses muscles et sa force brute, une autre un raisonnement construit et bien préparé, et une dernière mettra a profit sa capacité à se repérer et à communiquer avec les créatures hostiles présentes dans ces milieux, par exemple. Pour séduire, une personne peut miser sur le physique, l'intellect ou la parole. Etc.

Un score normal varie entre 1 (très faible) et 6 (exceptionnel), avec une moyenne autour de 3.

Les Compétences

Les niveaux de réussite critériée objectivement dans une Compétence (juste compétence) décrivent la maîtrise de chaque personne sur un type d'activité précis. Ces compétences s'entraînent et s'aiguisent avec l'expérience.

Chaque compétence est associée à un indicateur. Voici la liste des compétences communes (hors Plans et Sphères) :

- Corps :
  - Prouesse : les prouesses physiques (sprint sur 100 m, courir un marathon, porter le contenu d'une bibliothèque, ouvrir les portes coincées des archives...).
  - Esquive : la capacité à éviter physiquement les problèmes (chutes de pans de toit, globe céleste lancé avec force, maître occupé ne regardant pas devant soi...).

- Esprit :
  - Conception : la manipulation mentale d'éléments abstraits dans le but de répondre à un problème (créer un algorithme, mettre en équation, concevoir des classes...).
  - Synthèse : le regroupement de données et l'expression de la solution du problème (résoudre une équation, résumer un article, écrire un rapport...).
- Sens :
  - Analyse : collecter ce qui est vu, entendu, perçu pour avoir une idée concrète de la situation (chercher quelqu'un dans une pièce, repérer une inflexion dans la voix, lire un énoncé de SAÉ...).
  - Expression : préparer ses mots, son attitude pour exprimer concepts, idées, arguments ou émotions et les présenter à une autre personne (parler à des gens, écrire à des gens...).
- Maximum :
  - Survie : savoir gérer un passage dans un milieu hostile (blizzard, salle des profs...).
  - Séduction : user de ses charmes à fins de faciliter une interaction sociale.

Chaque compétence est associée à un niveau. En prenant l'exemple d'une compétence de Karate (caricaturale) :

1. Niveau Incompétent : les personnes qui n'ont jamais fait aucun sport.
2. Niveau Amateur : une personne qui a vu des films de Karate et a fait quelques assouplissements il y a dix ans ; après tout, ça ne doit pas être si compliqué que ça de taper dans une brique, non ? ...aïe.
3. Niveau Apprenti : une personne inscrite en club, pratiquant régulièrement, ceinture blanche à verte.
4. Niveau Compagnon : une personne inscrite en club, qui fait des compétitions, ceinture noire nidan environ.
5. Niveau Maître : il n'y en a qu'une dans le Dojo, c'est la personne qui fait les cours, on l'appelle Maître.
6. Niveau Agrégé : *Aujourd'hui, nous recevons exceptionnellement X-Sensei, roku-dan, qui vient vous parler des championnats de France...*
7. Niveau Ministre : quelqu'un qui est connu de toutes et tous dans le monde, même des non-spécialistes. Ici, Bruce Lee.

Création

Créer un personnage peut se faire avec des points de base ou des jets de dés, mais on préfèrera procéder comme suit.

1. Choisir le Plan et la Sphère du Mystère. Noter les compétences correspondantes.
2. Le Plan privilégie un des indicateurs : s'il s'agit du Corps, de l'Esprit ou du Sens, attribuer deux points à cette valeur, zéro aux autres ; s'il s'agit du maximum, choisir deux indicateurs parmi ces trois à placer à un, zéro au dernier.
3. Répartir les trois valeurs suivantes de plus aux indicateurs : deux, trois, et quatre. On obtient ainsi au moins deux, au plus six dans ces trois valeurs, et un total de onze points. Le Maximum est toujours la plus grande des trois autres valeurs.
4. La Sphère donne également un indicateur. Dans les compétences :
    a) Les quatre compétences de l'indicateur concerné reçoivent les valeurs suivantes, à répartir au choix : Amateur, Apprenti, Compagnon, Compagnon,
    b) Pour chacun des trois autres indicateurs, elles reçoivent les valeurs suivantes à répartir au choix : Incompétent, Amateur, Apprenti, Apprenti.
5. On dispose ensuite de trois augmentations libres de compétences, qui servent chacune à un de ces trois usages :
    a) choisir une compétence non encore augmentée et l'augmenter d'un niveau,
    b) choisir une compétence à Amateur et la passer à Incompétent, puis choisir une compétence non encore augmentée de niveau Apprenti ou inférieur, et l'augmenter de deux niveaux,
    c) choisir deux compétences à Incompétent à les passer à Amateur.
6. L'ensemble des compétences est alors entre Incompétent et Maître.
7. La santé mentale du personnage est égale au plus petit des trois indicateurs, multiplié par 10.

Une fois cette phase terminée, le personnage passe en *formation*.

Formation

En *formation*, un personnage suit sa vie et subit des *épreuves* (section suivante), qui lui rapportent de l'*expérience*. Ces *points d'expérience* peuvent être utilisés pour augmenter le niveau de ses indicateurs (par un entraînement impliquant des exercices intenses) ou de ses compétences (par l'étude et l'approfondissement de leur théorie et pratique).

La formation ne permet d'augmenter que d'un niveau à la fois. Pour passer au niveau supérieur, il faut la valeur numérique de ce niveau pour les compétences, et deux fois la valeur numérique de ce niveau pour les indicateurs. De plus, il n'est jamais possible de dépasser le seuil maximum, qui est de 7 pour les indicateurs et 6 (Ministre) pour les compétences.

Épreuves

Une *épreuve* consiste à utiliser une compétence par le biais d'un jet de dés à six faces ; le nombre de dés à lancer est égal à l'indicateur dont la compétence relève. Le total est à interpréter suivant le niveau de la compétence, en utilisant le tableau suivant :

Niveau	**Remplacer les dés affichant...**	**Par la valeur...**	**Ajouter tous les dés et...**
Incompétent	1, 2, 3, 4, 5, 6	0	0
Amateur	1	0	0
Apprenti			5
Compagnon	6	10	5
Maître	5, 6	10	10
Agrégé	4, 5, 6	10	15
Ministre	4, 5, 6	20	20

La valeur numérique totale ainsi calculée doit ensuite être convertie en *difficulté atteinte*. Dans l'exemple d'un sprint :

Difficulté atteinte si...	**Score $\geq$ ...**	**Interprétation**
Rien	0	n'a pas atteint la ligne de départ
Trivial	5	est arrivé à la ligne des 100 m
Facile	10	le 100 m en une minute
Normal	15	le 100 m en 20 secondes
Complexe	20	le 100 m en 15 secondes
Délicat	25	le 100 m en 12,5 secondes
Défi	30	le 100 m en 10 secondes
Exploit	50	le 100 m en 9,5 secondes
Surhumain	70	le 100 m en 9 secondes
Impossible	100	le 100 m en une seconde

Une *épreuve* est toujours associée à une *compétence*. Quand cette dernière est non disponible, elle peut être remplacée par celle de son choix du même indicateur, mais en abaissant la réussite (difficulté) obtenue de deux niveaux.

Une épreuve peut être *ouverte*, auquel cas il s'agit simplement de connaître la difficulté atteinte, ou *fermée*, avec une difficulté imposée. Dans ce dernier cas, les dés ne sont pas lancés si l'épreuve est trop difficile ou trop facile pour le niveau de compétence ciblé, auquel cas l'échec ou la réussite deviennent automatiques. Voir le tableau suivant :

Compétence	Réussite automatique	Échec automatique
Incompétent	Aucune	Toutes
Amateur	Aucune	Exploit et supérieures
Apprenti	Trivial	Surhumain et supérieures
Compagnon	Trivial	Impossible
Maître	Facile et inférieures	Aucune
Agrégé	Normal et inférieures	Aucune
Ministre	Complexe et inférieures	Aucune

Une épreuve fermée réussie donne un point d'expérience (section précédente) et peut restaurer un point de santé mentale (section suivante), une épreuve fermée échouée fait perdre un point de santé mentale (section suivante).

Santé Mentale

La *santé mentale* correspond à la résistance à l'adversité du personnage. Cette résistance dépend de son équilibre entre corps, esprit et sens, et se calcule donc à la création comme étant $10 \times m$, avec m le minimum entre ces trois valeurs. Cette donnée représente le *nombre de points de santé mental maximum* du personnage, qui n'évoluera pas.

Les *points actuels de santé mentale* varient du maximum en question à 0. Le personnage perd un point de santé mentale s'il échoue à une épreuve fermée. Si la santé mentale n'est pas à son maximum, elle peut être restaurée lors d'une épreuve fermée réussie, mais uniquement si l'épreuve fermée précédente était déjà réussie : il faut donc aux moins deux réussites pour compenser un échec.

Si la santé mentale arrive à 0, le personnage est *en crise* : le mystère perd ses moyens face à l'hostilité ambiante. Un personnage en crise ne peut plus utiliser de points de talent, et la difficulté passée par le personnage est décalée de deux niveaux vers le bas. Pour ne plus être en crise, le personnage doit regagner au moins un point de santé mentale, et donc réussir au moins deux épreuves fermées consécutives malgré ce handicap.

On peut également dépenser 10 points d'expérience pour restaurer un point de santé mentale, à condition de ne pas être en crise.

Talent

Les épreuves peuvent être difficiles. Les Mystères comptent parfois sur une ressource mystérieuse, le Talent, pour les réussir malgré cette difficulté.

Les points de Talent commencent à 10 et se dépensent au moment de passer une épreuve.

Voici leurs effets (un seul effet possible par point de Talent dépensé, et un seul point de Talent peut être dépensé par épreuve ; il n'est pas possible de revenir sur l'utilisation d'un point de Talent) :

1. *Trop facile !* : pour les épreuves fermées uniquement, de difficulté comprise entre *Complexe* et *Défi* : la difficulté est considérée comme *Facile* – dépense à effectuer avant le jet de des.
2. *Rattrapage* : pour toute épreuve, dépense à effectuer après le jet de dés : on relance les dés.
3. *Inspiration* : pour toute épreuve : ajoute 10 à tous les autres bonus au jet – dépense à effectuer avant le jet de dés.
4. *Au talent* : pour toute épreuve : on ne lance pas les dés, le résultat est *Normal* – dépense à effectuer avant le, et à la place du, jet de dés.

Un point de Talent dépensé peut être racheté pour k points d'expérience. k commence à 1 et est *doublé* à chaque rachat.

Annexe C

Feuille à remplir

(Ceci est un exemple, suivez votre *design*.)

